

AUTONOMOUS SOFTWARE EXPERIMENTS ON HERA: ANNEX E - EXAMPLE, AFC IMAGE ACQUISITION & SMART COMPRESSION

Reference	ESA-HERA-TECS-MAN-2026-002054
Issue/Revision	1.0
Date of Issue	19/06/2026

This example demonstrates how to acquire a scientific image from the Hera Asteroid Framing Camera (AFC) and process it to reduce data volume before storage.

The core logic uses a Smart RLE (Run-Length Encoding) algorithm. RLE is a lossless compression method, meaning the reconstructed image is identical to the original. The algorithm works line-by-line (1020 pixels at a time) compressing the line. If the compressed result is smaller than the original it will save the compressed line, otherwise it will save the original raw line.

Code Implementation:

```

1 #include "hera_interface.h"
2 #include "stub_utils.h"
3 #define APID_HERA      1500
4 #define TYPE_HERA      230
5 #define SUBTYPE_HERA   1
6 #define MAX_LINE_WIDTH 1020
7 #define MAX_LINES      1020
8
9 /**
10 * @brief Compresses an image line-by-line using smart RLE.
11 * * Logic:
12 * 1. Attempt to compress the line.
13 * 2. If compressed size < 1020 bytes, store the compressed data.
14 * 3. If compressed size >= 1020 bytes, store the original raw data.
15 * * @param img_data Pointer to the start of the raw image buffer.
16 * @param width Image width in pixels.
17 * @param height Image height in pixels.
18 */
19 void compress_image(const uint8* img_data, int width, int height) {
20     // Buffer size allocated for worst-case RLE expansion (2x)
21     // We allocate this locally to avoid corrupting the source image.
22     uint8 compressed_line[MAX_LINE_WIDTH * 2];
23
24     for (int y = 0; y < height; y++) {
25         // Pointer to the start of the current raw line in memory
26         const uint8* raw_line = &img_data[y * width];
27
28         int rle_index = 0;
29         int x = 0;
30         // --- Step 1: Attempt Compression ---
31         while (x < width) {
32             uint8 pixel_val = raw_line[x];
33             uint8 run_length = 1;
34             x++;
35             // Count identical consecutive pixels (up to 255)
36             while (x < width && raw_line[x] == pixel_val && run_length < 255) {
37                 run_length++;
38                 x++;

```

```

39     }
40     // Write the (Count, Value) pair to our temporary buffer
41     compressed_line[rle_index++] = run_length;
42     compressed_line[rle_index++] = pixel_val;
43     // If the compressed version is already bigger than the original,
44     // we can stop processing this line immediately to save CPU cycles.
45     if (rle_index >= width) {
46         break;
47     }
48 }
49
50 // --- Step 2: Compare & Report ---
51 if (rle_index < width) {
52     // Success: The compressed version is smaller. Store it.
53     Hera_Science_Report(APID_HERA, TYPE_HERA, SUBTYPE_HERA, (void*)
compressed_line, (uint16)rle_index);
54 } else {
55     // The compressed version is larger or equal. Store the original raw
line.
56     Hera_Science_Report(APID_HERA, TYPE_HERA, SUBTYPE_HERA, (void*)
raw_line, (uint16)width);
57 }
58 }
59 }
60
61 int main(void) {
62     error_code_t status;
63     uint32 exposure_time = 500;
64     print_str("\n--- HERA RLE Test ---\n");
65     // 1. Request Core0 to acquire an image from the camera
66     print_str("Acquiring AFC...\n");
67     status = Hera_AFC_AcquireSingleImage(exposure_time);
68
69     if (status == HERA_OK) print_str(">> OK: Acquired.\n");
70     else print_str(">> FAIL.\n");
71     // 2. Retrieve the memory address of the new image
72     uint8* pImg = Hera_AFC_GetImageBuffer();
73
74     // 3. Run the compression and storage routine
75     compress_image(pImg, MAX_LINE_WIDTH, MAX_LINES);
76     // 4. Send a Housekeeping (HK) report with e.g. summary of results
77     print_str("HK Report...\n");
78     uint8 d[] = {0xCA, 0xFE};
79     Hera_HK_Report(99, d, 2);
80
81     print_str("--- TEST END ---\n");
82
83     while(1);

```



```
84     return 0;  
85 }
```